

SPARQL

IE650 Knowledge Graphs



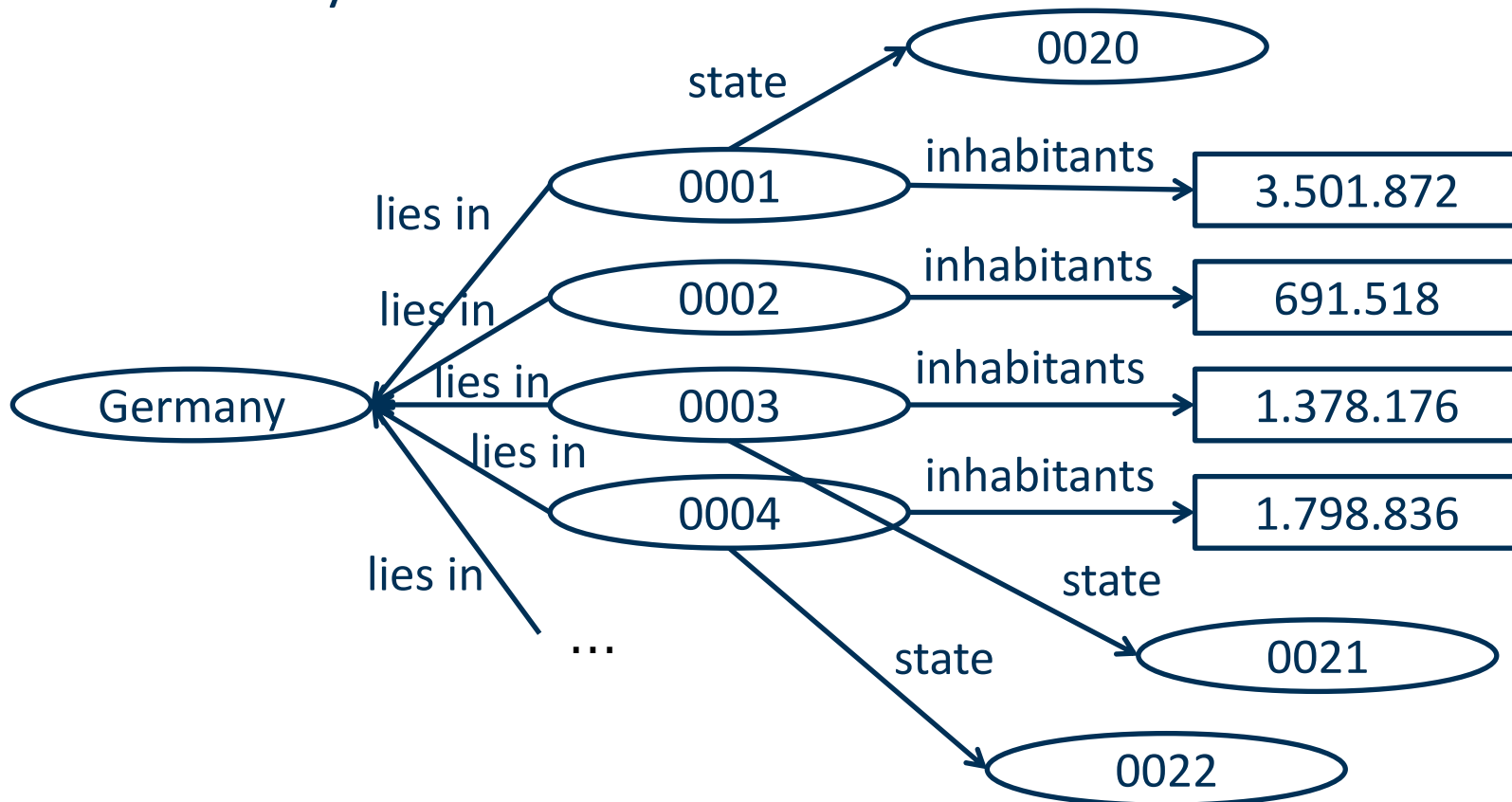
Previously on “Knowledge Graphs”

- We have got to know
 - The RDF and RDFS languages
 - The Linked Open Data paradigm
- We have accessed KGs and Linked Open Data
 - With browsers and via programming frameworks
 - Jumping from node to node in the graph
- ...let us have a closer look!



An Example RDF Graph

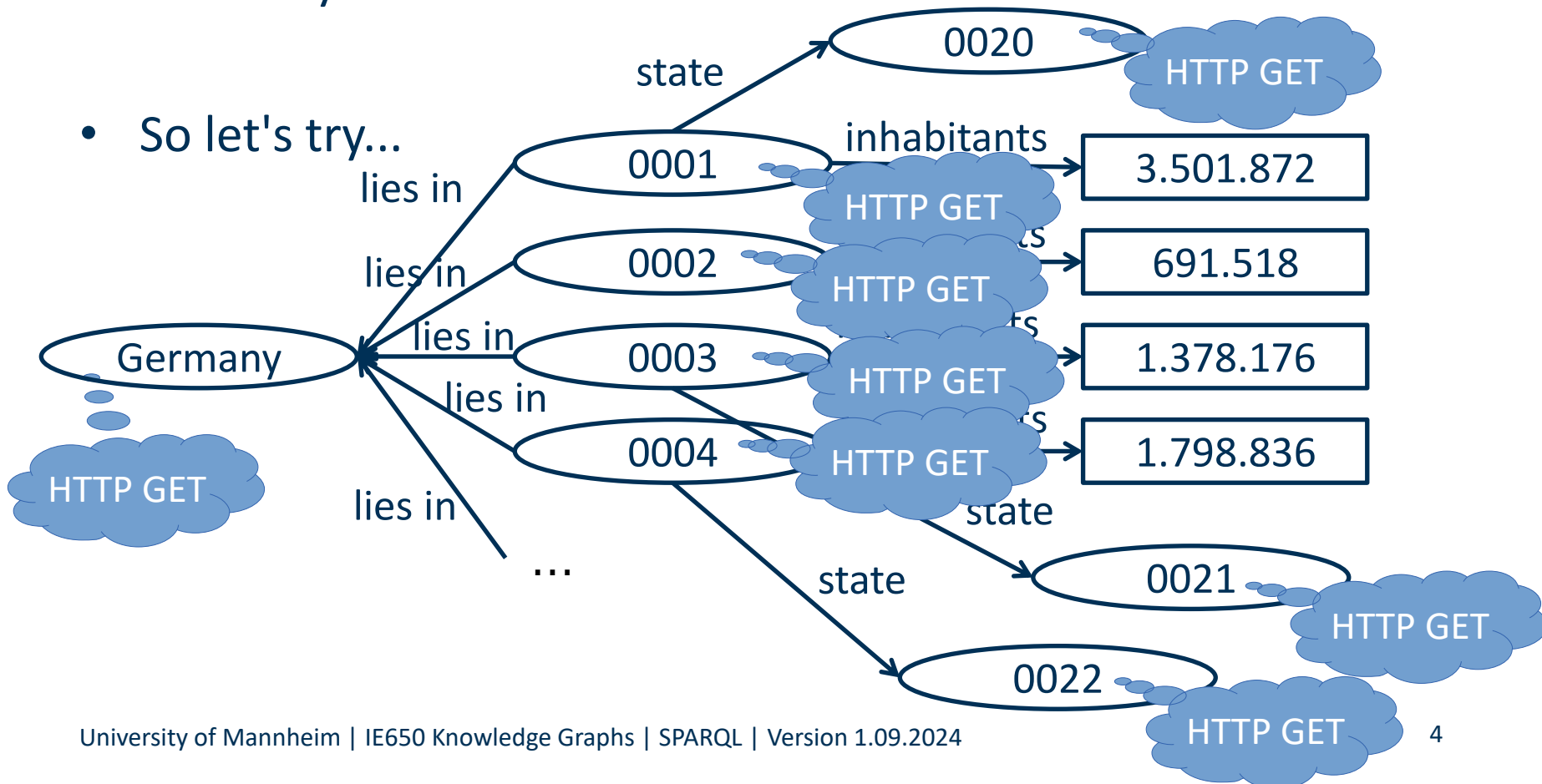
- Question: In which states are the five biggest cities of Germany located?



Information Retrieval on Linked Open Data

- Question: in which states are the five biggest cities of Germany located?

- So let's try...



Information Retrieval on Linked Open Data

- Observations
 - Navigation across derefencable URIs ultimately leads to a goal
 - But it is tedious
 - A lot of useless data is potentially retrieved
- Different information needs
 - Good for retrieving simple facts
 - Less efficient for more complex questions

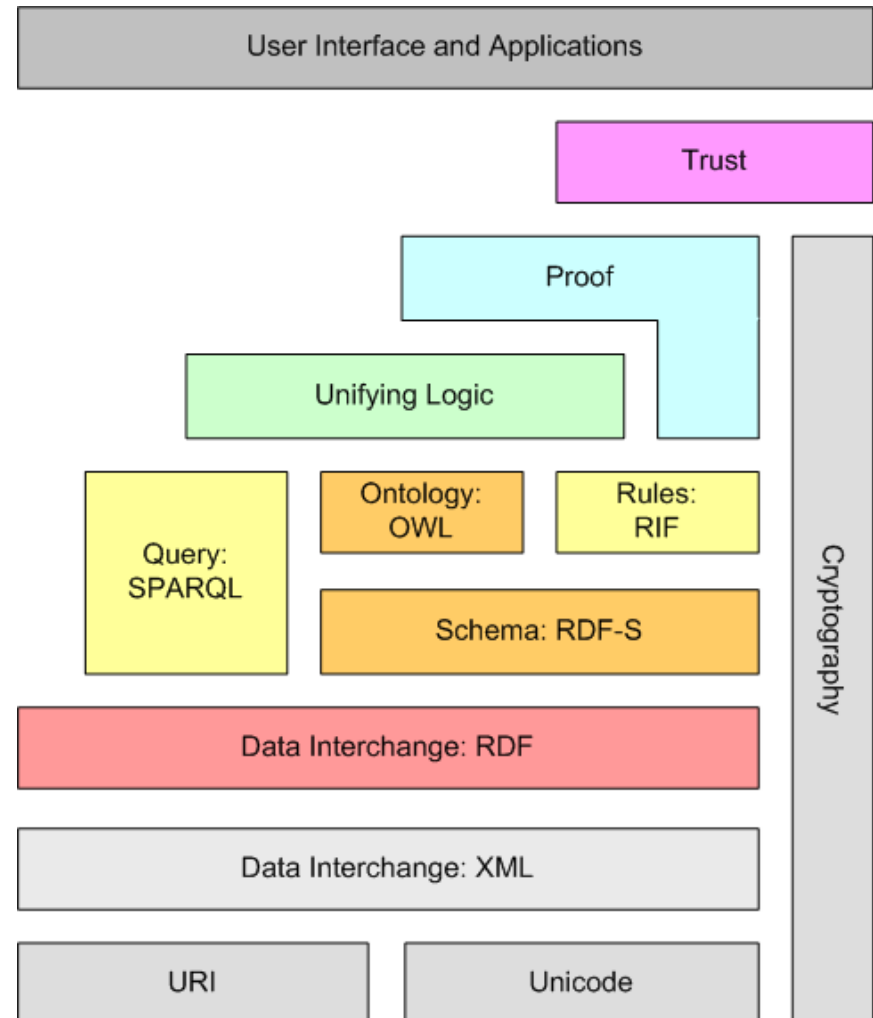
Semantic Web Technology Stack



here be dragons...

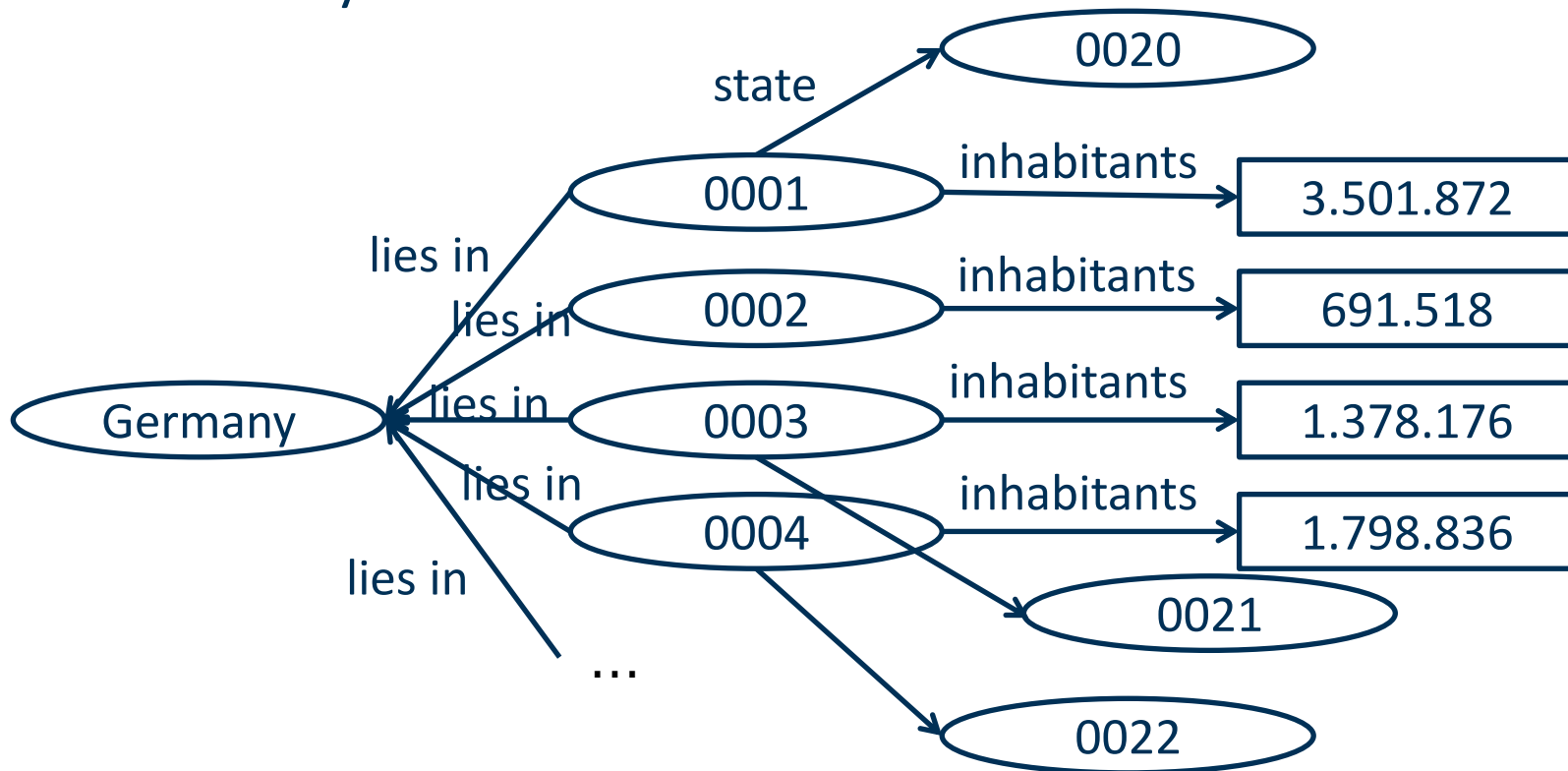
Knowledge Graph Technologies
(This lecture)

Technical
Foundations



What Would We Like to Have?

- Question: in which states are the five biggest cities of Germany located?



Wanted: A Query Language for the KGs

- ...just like SQL is for relational databases

```
SELECT    name, birthdate FROM customers
WHERE     id = '00423789'
```

id	name	birthdate
00183283	Stephen Smith	23.08.1975
00423782	Julia Meyer	05.09.1982
00789534	Sam Shepherd	31.03.1953
00423789	Herbert King	02.04.1960
...	...	...

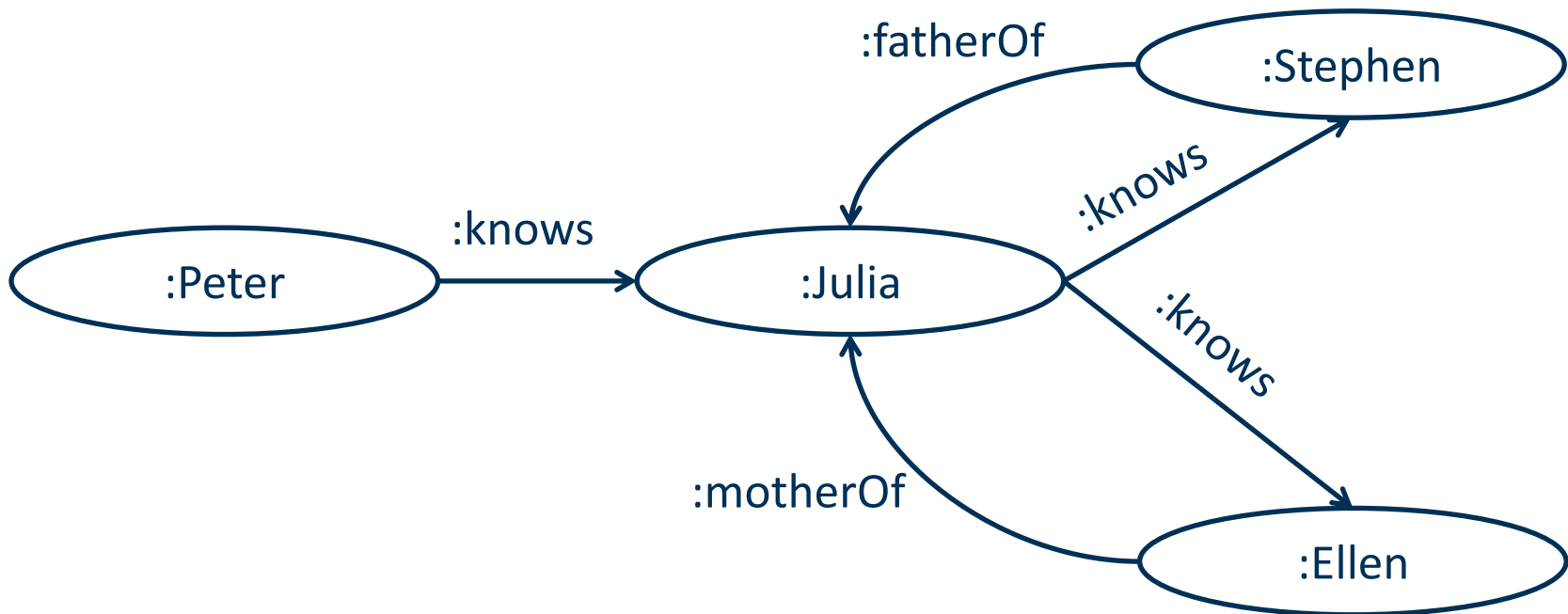
Wanted: A Query Language for the KGs

- SPARQL: "SPARQL Query Language for RDF"
 - a recursive acronym
- A W3C Standard since 2008
- Allows for querying RDF graphs



SPARQL: General Idea

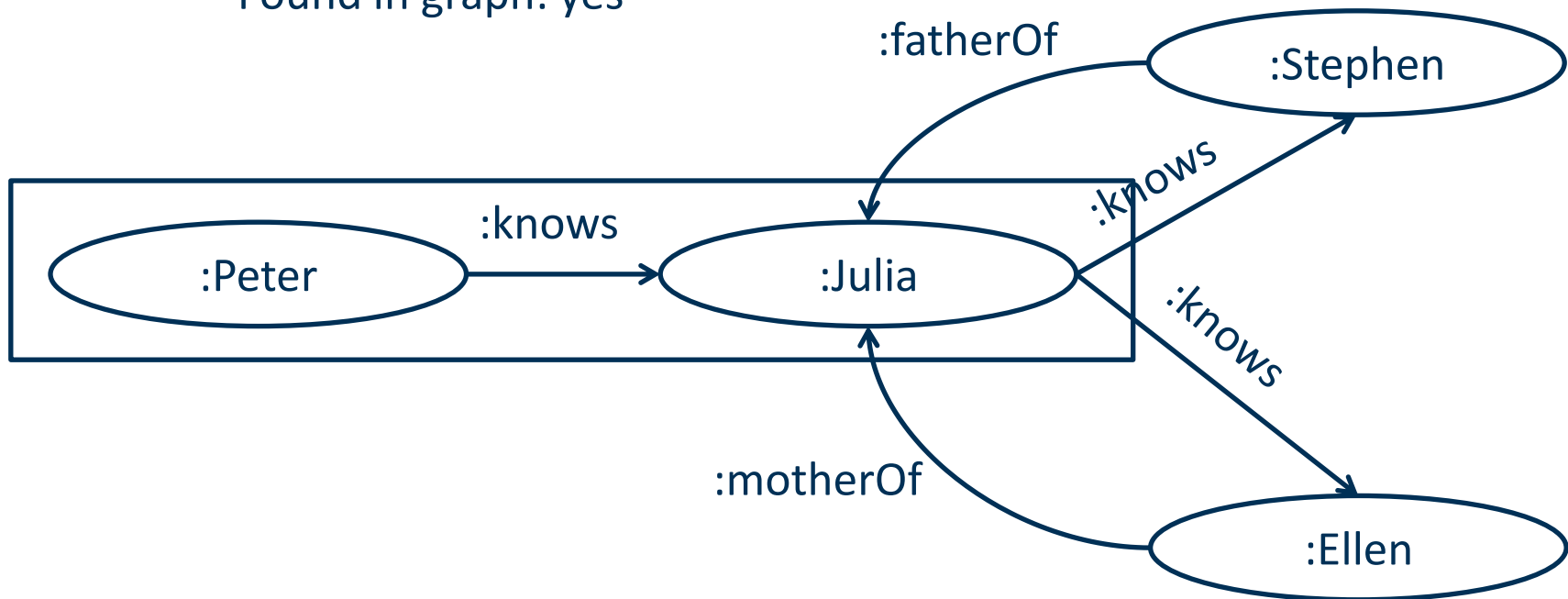
- Look for specific subgraphs in a knowledge graph
 - Subgraphs may contain variables



SPARQL Basics:

Graph Pattern Matching

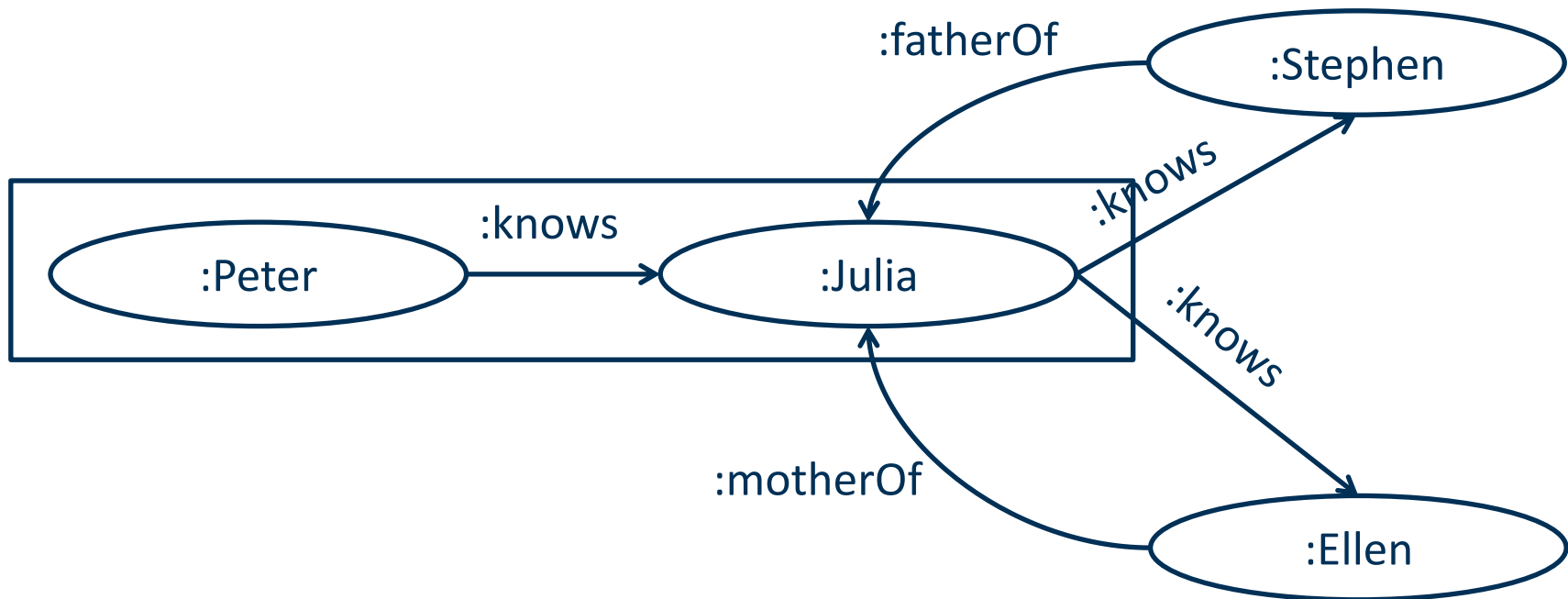
- Subgraph: :Peter :knows :Stephen
 - Found in graph: no
- Subgraph: :Peter :knows :Julia .
 - Found in graph: yes



SPARQL Basics:

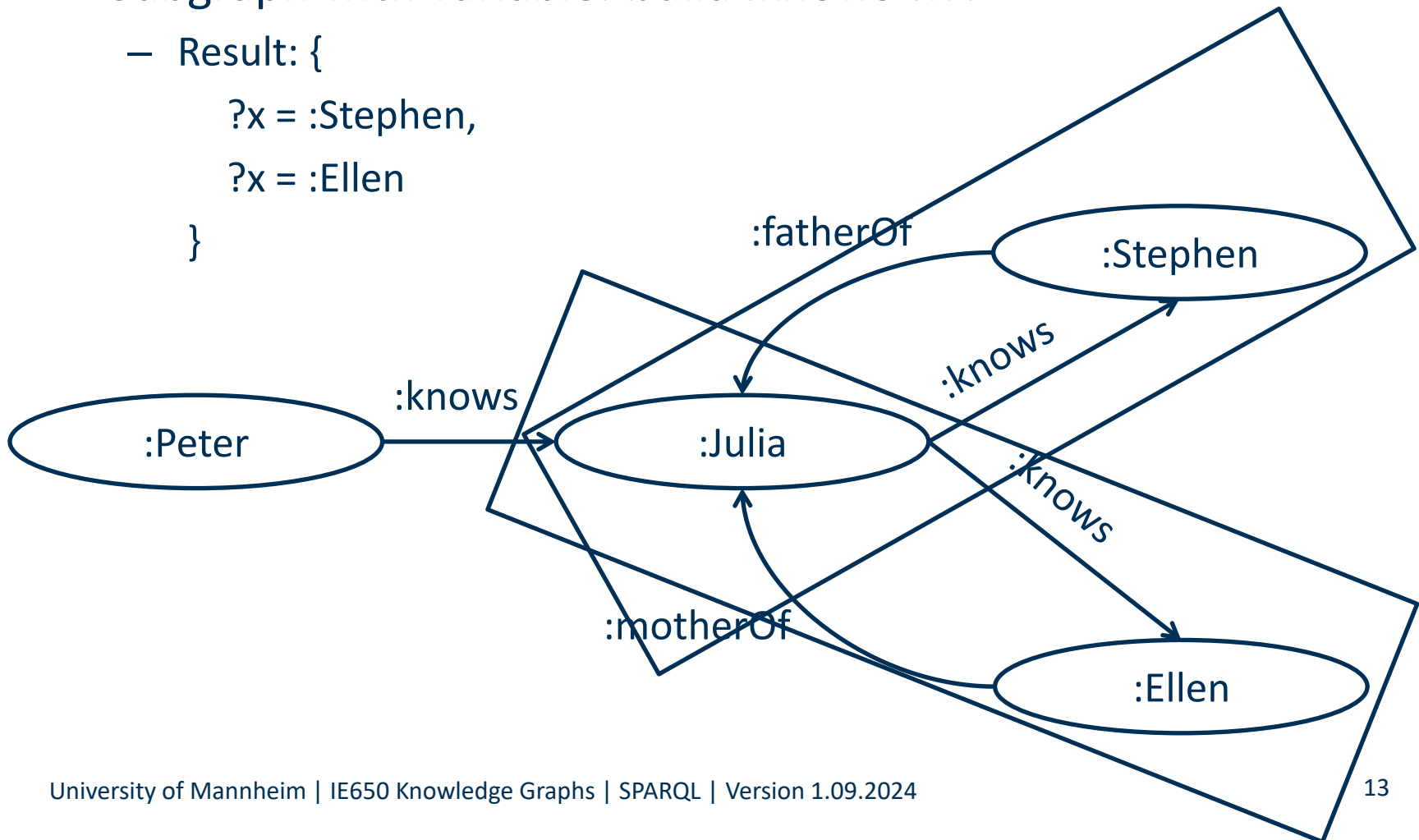
Graph Pattern Matching

- Subgraph with variable: `?x :knows :Julia .`
 - Result: `{ ?x = :Peter }`



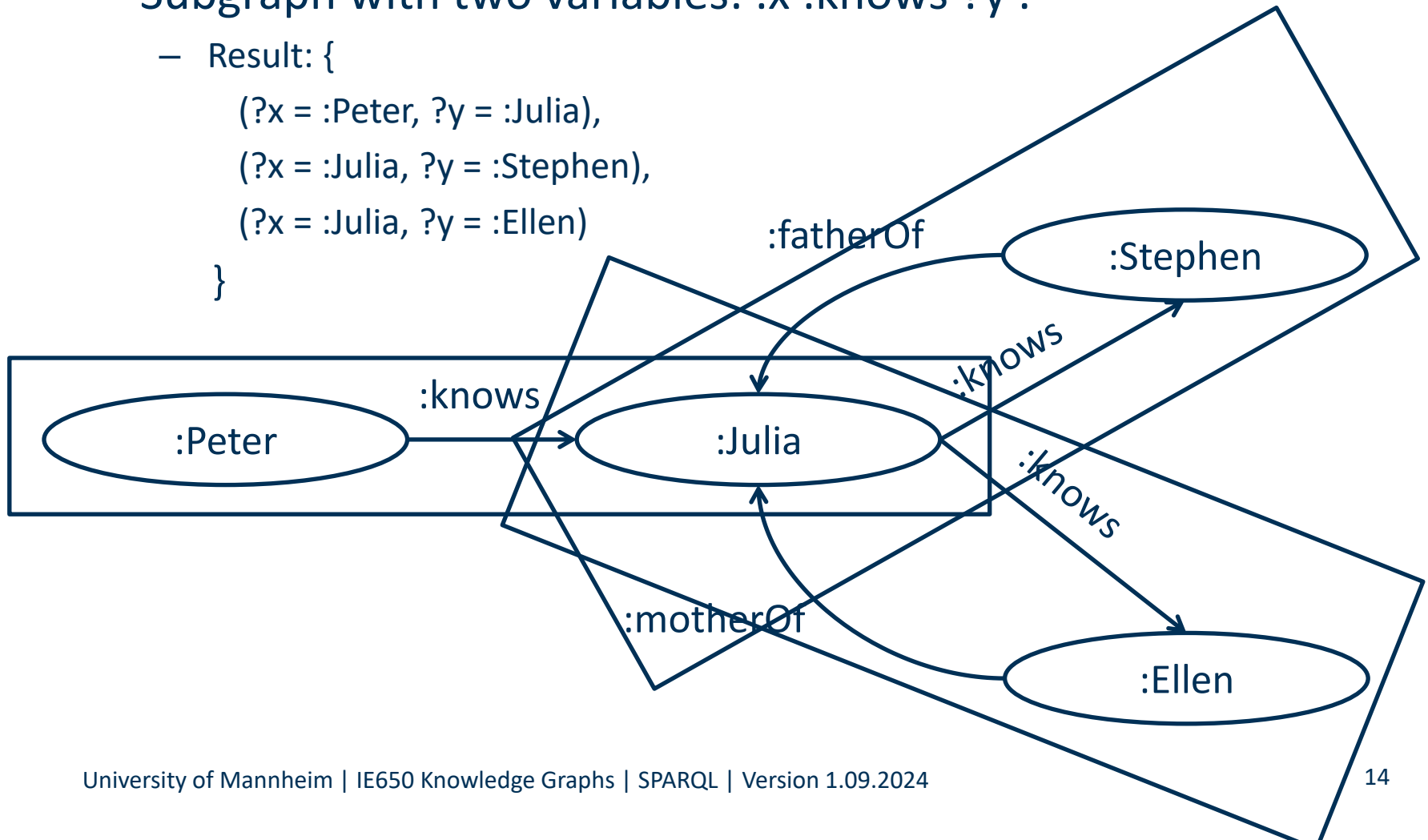
SPARQL Basics: Graph Pattern Matching

- Subgraph with variable: `:Julia :knows ?x .`
 - Result: {
 - `?x = :Stephen,`
 - `?x = :Ellen`



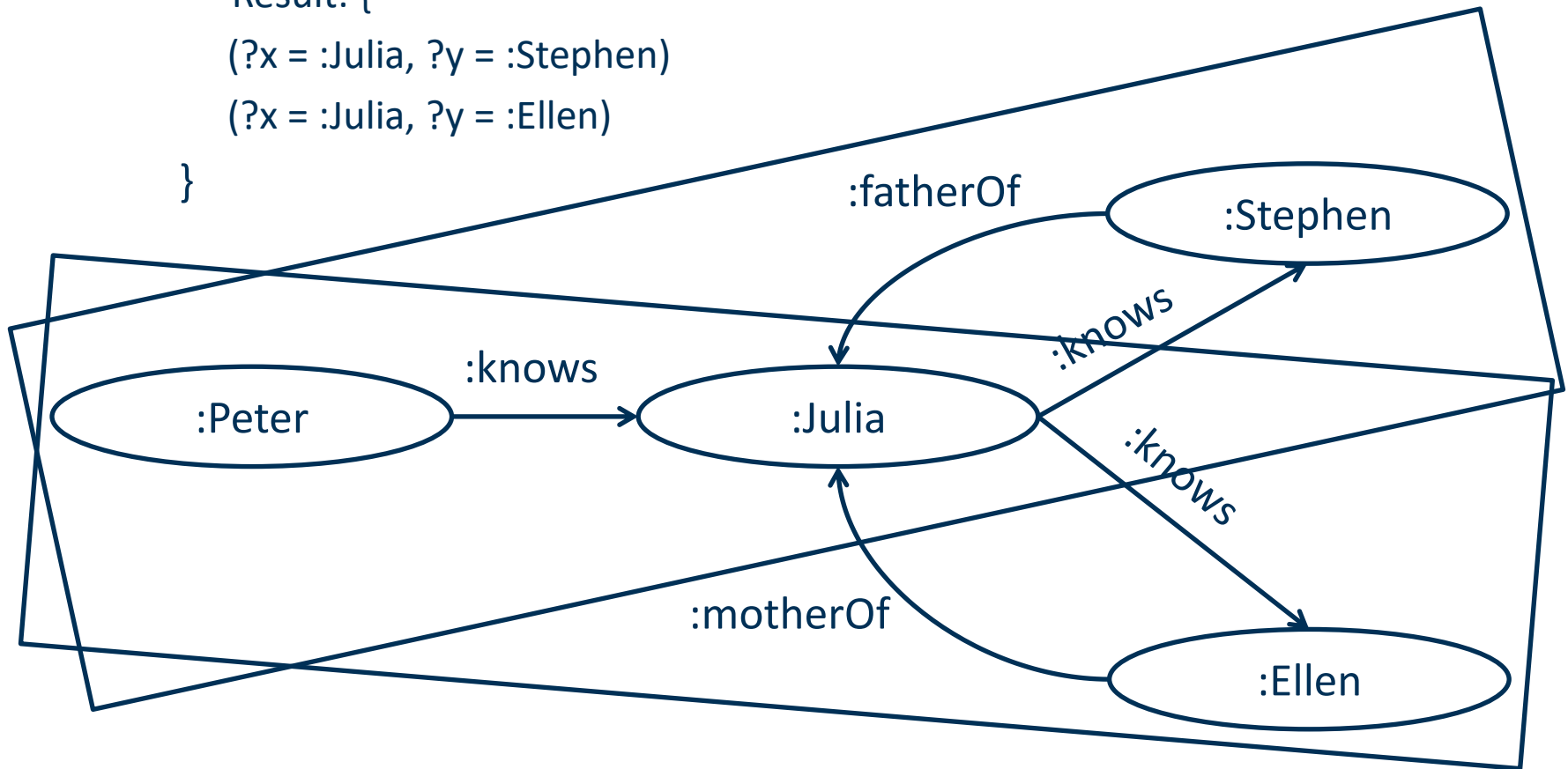
SPARQL Basics: Graph Pattern Matching

- Subgraph with two variables: `:x :knows ?y`.
 - Result: {
 (`?x = :Peter, ?y = :Julia`),
 (`?x = :Julia, ?y = :Stephen`),
 (`?x = :Julia, ?y = :Ellen`)
}



SPARQL Basics: Graph Pattern Matching

- Complex subgraph: `:Peter :knows ?x . ?x knows ?y .`
 - Result: {
(?x = :Julia, ?y = :Stephen)
(?x = :Julia, ?y = :Ellen)
}



Hello SPARQL!

- Example:

```
SELECT ?child  
WHERE { :Stephen :fatherOf ?child }
```

Expressions with ?
denote variables



SPARQL Syntax

- Basic structure:

```
SELECT <list of variables>  
WHERE { <pattern> }
```

- Variables denoted with ?

- Prefixes as in RDF/N3:

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>  
SELECT ?person ?name  
WHERE { ?person foaf:name ?name }
```

- The <pattern> in the WHERE clause is like N3
 - with variables

```
{?p foaf:name ?n }
```

```
{?p foaf:name ?n; foaf:homepage ?hp }
```

```
{?p foaf:knows ?p1, ?p2 }
```

SPARQL: Pattern Matching on RDF Graphs

- A person who has a daughter and a son:

```
{ ?p :hasDaughter ?d ; :hasSon ?s . }
```
- A person knowing two persons who know each other

```
{ ?p :knows ?p1 , ?p2 . ?p1 :knows ?p2 . }
```
- A person who has two children:

```
{ ?p :hasChild ?c1, ?c2 . }
```

SPARQL: Pattern Matching on RDF Graphs

- ~~A person who has two children:~~

~~{ ?p :hasChild ?c1, ?c2 . }~~

Observation: different variables are not necessarily bound to different resources!

- ResultSet:

– ?p=:Stephen, ?c1=:Julia, ?c2=:Julia



SPARQL: Blank Nodes

- WHERE clause: an RDF graph with variables

```
SELECT ?person1 ?person2 ?otherPerson
WHERE {
    ?person1 :knows ?otherPerson .
    ?otherPerson :fatherOf ?person2 . }
```

- Result:

- ?person1 = :Peter, ?person2 = :Julia; ?otherPerson = _:x1

- Note: Blank Node IDs are only unique within one result set!



SPARQL: Matching Literals

- Strings

```
{ ?country :name "Germany" . }
```

- Watch out for language tags!

```
{ ?country :name "Germany"@en . }
```

→ The Strings "Germany" and "Germany"@en are different!

- Numbers:

```
{ ?person :age "42"^^xsd:int . }
```

- Short hand notation:

```
{ ?person :age 42 . }
```

SPARQL: Filters

- Used for further restricting results

```
{?person :age ?age . FILTER(?age < 42) }
```

- Operators for comparisons:

= != < > <= >=

- Logical operations:

& & | | !

SPARQL: Filters

- Persons with younger siblings

```
{ ?p1 :siblingOf ?p2 .  
  ?p1 :age ?a1 .  
  ?p2 :age ?a2 .  
  FILTER(?a2 < ?a1) }
```

- Persons that have both younger and older siblings

```
{ ?p1 :siblingOf ?p2, ?p3 .  
  ?p1 :age ?a1 .  
  ?p2 :age ?a2 .  
  ?p3 :age ?a3 .  
  FILTER(?a2 < ?a1 && ?a3 > ?a1) }
```

Question: why do we get
different persons for p2 and p3?

SPARQL: Filters

- Second try: a person with two children

```
{ ?p :hasChild ?c1, ?c2 . FILTER( ?c1 != ?c2) }
```
- A slight improvement
→ Variables are now bound to different resources
- But: we still have the Non-Unique Naming Assumption
→ i.e., given that
:Peter :hasChild :Julia .
:Peter :hasChild :Stefan .
we still cannot conclude that Peter has two children!
- Furthermore, there is still the Open World Assumption
→ i.e., Peter could also have more children

Filters for Strings

- Searching in Strings: using regular expressions

- People called “Ann”

```
{?person :name ?n . FILTER(regex(?n, "^Ann$")) }
```

```
{?person :name ?n . FILTER(regex(?n, "Ann")) }
```

→ the second variant would also find, e.g., “Mary-Ann”

- `str`: URIs and Literals as strings
- allows for, e.g., searching for literals across languages

```
{?country :name ?n . FILTER(str(?n) = "Tyskland") }
```

Further Built-In Features

- Querying the type of a resource
 - `isURI`
 - `isBLANK`
 - `isLITERAL`
- Querying for the data type and language tags of literals
 - `DATATYPE(?v)`
 - `LANG(?v)`
- Comparing the language of two literals
 - `langMATCHES(?v1, ?v2)`
 - **Caution:** given `?v1 = "Januar"@DE, ?v2 = "Jänner"@DE-at`
`LANG(?v1) = LANG(?v2) → false`
`langMATCHES(?v1, ?v2) → true`

Combining Patterns

- Find the private and work phone number

```
      { ?p :privatePhone ?nr }  
UNION { ?p :workPhone ?nr }
```

- UNION creates a set union

```
?p = :peter, ?nr = 123;  
?p = :john, ?nr = 234;  
?p = :john, ?nr = 345;  
...
```

That happens if John has both a private and a work phone

Optional Patterns

- Find a person's phone number and fax number, **if existing**

```
        { ?p :phone ?tel }  
OPTIONAL { ?p :fax ?fax }
```

- OPTIONAL also creates unbound variables

?p = :peter, ?tel = 123, ?fax = 456;

?p = :john, ?tel = 234, ?fax = ;

?p = :julia, ?nr = 978; ?fax = 349;

...

Unbound variable:
John does not have a fax
number (as far as we know)

Sorting and Paging Results

- **Sorting:** `ORDER BY ?name`
- **Limitations:** `LIMIT 100`
- **Lower Bounds:** `OFFSET 200`
- **Example: persons 101-200, ordered by name**
 - `ORDER BY ?name LIMIT 100 OFFSET 100`
- **LIMIT/OFFSET without ORDER BY:**
 - Result orderings are not deterministic
 - There is no default ordering

Sorting and Paging Results

- Application scenarios:
 - Some SPARQL services limit their result set sizes
 - Pre-loading in applications
- Application example:
 - let the user browse cities
 - it is more likely that users want to see the big cities
 - display 100 biggest cities on one page, show more on demand
- ```
SELECT ?city ?population
WHERE {?city hasPopulation ?population}
ORDER BY DESC(?population)
LIMIT 100
```

# Filtering Duplicates

- ```
SELECT DISTINCT ?person
  WHERE { ?person :privatePhone ?nr }
  UNION{ ?person :workPhone ?nr }
```
- This means: all results with identical variable bindings are filtered
- This does not mean: persons identified by *?person* are actually different
- Why?
 - Non-unique naming assumption

Custom Built-Ins

- Some SPARQL engines allow special constructs
- also known as *Custom Built-Ins*
- Example: geographic processing
 - Knowledge Graph: DBpedia
 - Runs on Virtuoso

Custom Built-Ins

- Querying for coordinates

```
SELECT ?x
WHERE {
    ?x geo:long ?long;
        geo:lat ?lat .
    FILTER (?long>8.46 &&
            ?long<8.47 &&
            ?lat>49.48 &&
            ?lat<49.49)
}
```

SPARQL | HTML5 table

x

[http://dbpedia.org/resource/Action_at_Mannheim_\(1795\)](http://dbpedia.org/resource/Action_at_Mannheim_(1795))

http://dbpedia.org/resource/GESIS_-_Leibniz_Institute_for_the_Social_Sciences

http://dbpedia.org/resource/Reiss_Engelhorn_Museum

http://dbpedia.org/resource/University_of_Mannheim__University_of_Mannheim__1

http://dbpedia.org/resource/University_of_Mannheim

http://dbpedia.org/resource/University_of_Mannheim_Business_School

http://dbpedia.org/resource/Jesuit_Church,_Mannheim

[http://dbpedia.org/resource/Battle_of_Mannheim_\(1799\)](http://dbpedia.org/resource/Battle_of_Mannheim_(1799))

http://dbpedia.org/resource/Battle_of_Seckenheim

http://dbpedia.org/resource/Klasmühl'_am_Rathaus

<http://dbpedia.org/resource/Mannheim>

http://dbpedia.org/resource/Mannheim_Business_School

http://dbpedia.org/resource/Mannheim_University_Library

http://dbpedia.org/resource/Zentrum_für_Europäische_Wirtschaftsforschung

http://dbpedia.org/resource/Mannheim_Observatory

http://dbpedia.org/resource/Palais_Bretzenheim

[http://dbpedia.org/resource/Angel_of_Peace_\(Mannheim\)](http://dbpedia.org/resource/Angel_of_Peace_(Mannheim))

http://dbpedia.org/resource/Graduate_School_of_Economic_and_Social_Sciences

http://dbpedia.org/resource/Mannheim_Palace

http://dbpedia.org/resource/Mannheim_Palace_Church

Custom Built-Ins

- More complex queries
 - All museums within a 10km radius of a given point

```
SELECT ?x
WHERE { ?x rdf:type dbo:Museum;
        geo:long ?long;
        geo:lat ?lat
        FILTER (bif:st_intersects(
            bif:st_point(?long, ?lat),
            bif:st_point(8.466, 49.488), 10))
}
```

x

http://dbpedia.org/resource/Automuseum_Dr_Carl_Benz

http://dbpedia.org/resource/Reiss_Engelhorn_Museum

http://dbpedia.org/resource/Kunsthalle_Mannheim

<http://dbpedia.org/resource/Technoseum>

Further Query Types: ASK

- So far, we have only looked at SELECT queries
- ASK allows for yes/no queries:
 - e.g., are there persons with siblings?

```
ASK {?p :hasSibling ?s . }
```

- Often faster than SELECT queries
- The answer is true or false
 - *false* means: there are no matching sub graphs
 - do not misinterpret (Open World Assumption!)

Further Query Types: DESCRIBE

- All properties of a resource

```
DESCRIBE <http://dbpedia.org/resource/Berlin>
```

- Can be combined with a WHERE clause

```
DESCRIBE ?city WHERE { :Peter :livesIn ?city . }
```

- Allows for exploration of a dataset with unknown structure
- Caution: types of results are not standardized, results vary from implementation to implementation!



Further Query Types: CONSTRUCT

- Creates a new RDF graph

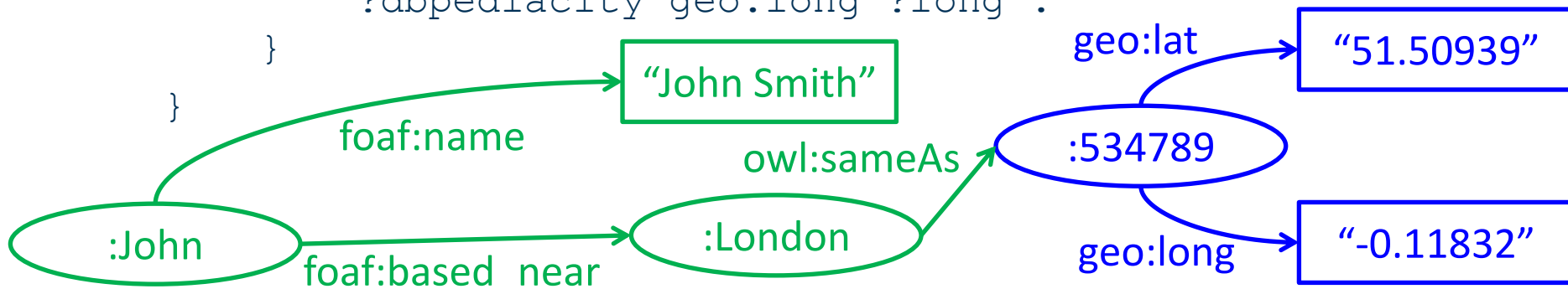
```
CONSTRUCT {  
    ?x rdfs:seeAlso <http://dbpedia.org/resource/Berlin> .  
}  
WHERE {  
    <http://dbpedia.org/resource/Berlin> ?y ?x .  
    FILTER (isURI(?x))  
}
```

- CONSTRUCT returns complete RDF graphs
 - e.g., for further processing

Query Federation

- Queries can be answered over *multiple* SPARQL endpoints
- Example

```
SELECT ?name ?lat ?long WHERE {
  ?x a foaf:Person .
  ?x foaf:name ?name .
  ?x foaf:based_near ?city .
  ?city owl:sameAs ?dbpediacity .
  SERVICE <http://dbpedia.org/sparql> {
    ?dbpediacity geo:lat ?lat .
    ?dbpediacity geo:long ?long .
  }
}
```



SPARQL: Wrap-Up

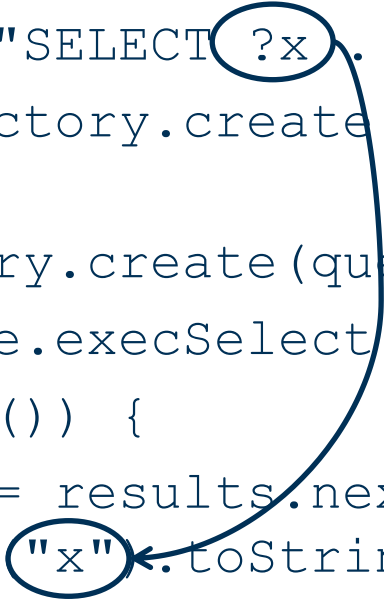
- SPARQL is a query language for the semantic web
- Basic principle: pattern matching on graphs
- SPARQL allows for directed search for information instead of navigating the graph from node to node
- Results follow the semantic principles of RDF!
 - Open World Assumption
 - Non-unique naming assumption



Example: Jena + SPARQL

- Querying models with SPARQL

```
String queryString = "SELECT ?x ...";
Query query = QueryFactory.create(queryString);
QueryExecution qe =
    QueryExecutionFactory.create(query, model);
ResultSet results = qe.execSelect();
while(results.hasNext()) {
    QuerySolution sol = results.next();
    String s = sol.get("x").toString();
    ...
}
```



SPARQL and Reasoning

- Important note:
 - By default, SPARQL does **not** use any reasoning
- Given the following T-box

```
:Museum rdfs:subClassOf :Building .
```
- and A-box

```
:Technoseum rdf:type :Museum .
```
- The result of

```
SELECT ?x WHERE {?x rdf:type :Building}
```

is empty!



SPARQL and Reasoning

- Important note:
 - By default, SPARQL does **not** use any reasoning
- Given the following T-box

```
:Museum rdfs:subClassOf :Building .
```
- and A-box

```
:Technoseum rdf:type :Museum .
```
- We would have to use something like

```
SELECT ?x WHERE {  
    {?x rdf:type :Building } UNION  
    {?x rdf:type ?t .  
     ?t rdfs:subClassOf :Building }  
}
```



Recap: Reasoning with Jena

- Given: a schema and some data

```
Model schemaModel = ModelFactory.createDefaultModel();  
InputStream IS = new  
    FileInputStream("data/example_schema.rdf");  
schemaModel.read(IS);
```

```
Model dataModel = ModelFactory.createDefaultModel();  
IS = new FileInputStream("data/example_data.rdf");  
dataModel.read(IS);
```

```
Model reasoningModel =  
    ModelFactory.createRDFSModel(schemaModel, dataModel);
```

- Now, reasoningModel contains all derived facts

Example: Jena + SPARQL + Reasoning

- Derived facts can also be queries with SPARQL
- Given the reasoningModel

```
Query q = QueryFactory.create(
    "SELECT ?t WHERE
    { <http://example.org/Madrid>
      <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>
        ?t .}" );
QueryExecution qexec =
    QueryExecutionFactory.create(q, reasoningModel);
ResultSet rs = qexec.execSelect();
while(rs.hasNext())
    String type = rs.next().get("t");
```

- Here, the query produces two solutions
 - `http://example.org/City`
 - `http://www.w3.org/2000/01/rdf-schema#Resource`

Accessing Public SPARQL Endpoints

- SPARQL Endpoints are an important building block of the Semantic Web tool stack
- Access using Jena:

```
String query = "SELECT ...";  
String endpoint = "http://dbpedia.org/sparql";  
Query q = QueryFactory.create(strQuery);  
QueryExecution qexec =  
    QueryExecutionFactory.sparqlService(endpoint, q);  
ResultSet RS = qexec.executeSelect();
```

Accessing Public SPARQL Endpoints

- Recap:
 - Jena uses the iterator pattern quite frequently
- Observation:
 - SPARQL ResultSets are also like iterators
 - Data can be retrieved from the server little by little

Triple Pattern Fragments

- Observation:
 - Operating SPARQL endpoints is costly
 - Hence, there are often downtimes
 - Maintenance often ends as project funding runs out
 - Accessing data via dumps or dereferencing is time consuming
 - See initial experiment
- Triple Pattern Fragments provide a middle ground solution



Triple Pattern Fragments

- Only allow simple restrictions
 - i.e., only {?s ?p ?o} (each can be a URI or a variable)
- Provide results in a paged fashion
 - Estimated count
 - Links to further pages

Data Portal @ linkeddatafragments.org

DBpedia 2014

Query DBpedia 2014 by triple pattern

subject: _____
predicate: <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>
object: <http://dbpedia.org/ontology/Restaurant>

Find matching triples

Matches in DBpedia 2014 for { ?s <<http://www.w3.org/1999/02/22-rdf-syntax-ns#type>> <<http://...>

Showing triples 1 to 100 of ±1,213 with 100 triples per page. [next](#)

```
&samhoud_places type Restaurant.  
't_Brouwerskolkje type Restaurant.  
't_Fornuis type Restaurant.  
't_Ganzenest type Restaurant.  
't_Koetshuis type Restaurant.  
't_Misverstant type Restaurant.  
't_Nonnetje type Restaurant.  
't_Schulten_Hues type Restaurant.  
't_Veerhuis type Restaurant.  
1321_Downtown_Taproom_Bistro type Restaurant.  
21_Club type Restaurant.  
36_on_the_Quay type Restaurant.  
5_North_St type Restaurant.  
68-86_Bar_and_Restaurant type Restaurant.  
Aan_de_Poel type Restaurant.  
Ad_Hoc_(restaurant) type Restaurant.  
Akelare type Restaurant.  
Alain_Ducasse_at_the_Dorchester type Restaurant.  
Albannach_(restaurant) type Restaurant.
```



Triple Pattern Fragments

- Most SPARQL queries can be solved by iteratively retrieving TPFs
 - Successively issuing new *selectors*
 - More targeted, i.e., less calls, than derefencing individual URIs

Query Linked Data on the Web **#LD**
Live in your browser, powered by Triple Pattern Fragments. Linked Data Fragments

Choose datasources:

Type or pick a query:

```
SELECT ?movie ?title ?name
WHERE {
  ?movie dbpedia-owl:starring [ rdfs:label "Brad Pitt"@en ];
  rdfs:label ?title;
  dbpedia-owl:director [ rdfs:label ?name ].
  FILTER LANGMATCHES(LANG(?title), "EN")
  FILTER LANGMATCHES(LANG(?name), "EN")
}
```

Execute query 43 results in 1.4s

Query results:

?movie	http://dbpedia.org/resource/12_Monkeys
?title	"12 Monkeys"@en
?name	"Terry Gilliam"@en
?movie	http://dbpedia.org/resource/A_River_Runs_Through_It_film
?title	"A River Runs Through It (film)"@en
?name	"Robert Redford"@en
?movie	http://dbpedia.org/resource/Across_the_Tracks
?title	"Across the Tracks"@en
?name	"Sandy Tung"@en
?movie	http://dbpedia.org/resource/Babel_film
?title	"Babel (film)"@en
?name	"Alejandro González Iñárritu"@en
?movie	http://dbpedia.org/resource/Burn_After_Reading

Execution log:

```
Requesting http://fragments.dbpedia.org/2016-04/en
Requesting http://fragments.dbpedia.org/2016-04/en?predicate=http%3A%2F%2Fwww.w3.org%2F2000%2F01%2F
Requesting http://fragments.dbpedia.org/2016-04/en?predicate=http%3A%2F%2Fwww.w3.org%2F2000%2F01%2F
```

Triple Pattern Fragments

- Example: astronauts born in capital countries

Select ?x ?y ?z where {

?x a dbpedia-owl:Astronaut . +/- 651

?x dbpedia-owl:birthPlace ?y . $\pm 1,137,061$

?z dbpedia-owl:capital ?y . $\pm 5,034$

?z a dbpedia-owl:Country $\pm 13,779$

}

- Algorithm:

- retrieve pattern: ?x a dbpedia-owl:Astronaut . 653
- for each result ?x retrieve: ?x dbpedia-owl:birthPlace ?y . 1,209
- for each result ?y retrieve: ?z dbpedia-owl:capital ?y . 637
- for each result ?z check: ?z a dbpedia-owl:Country .

Triple Pattern Fragments vs. SPARQL

- Middle ground between
 - setting up a SPARQL server (costly for the server)
 - providing a full RDF dump (costly for the client)
- In our example, a SPARQL query was broken down into ~3k HTTP GET requests
 - Using clever index structures, this might still be faster
 - Results may also be streamed – allows for early stopping



Triple Pattern Fragments vs. SPARQL

- All SPARQL constructs can be translated to a TPF query plan
- Some are quite fast
 - e.g., typical star-shaped queries
- Some are rather slow
 - e.g., regex queries for labels



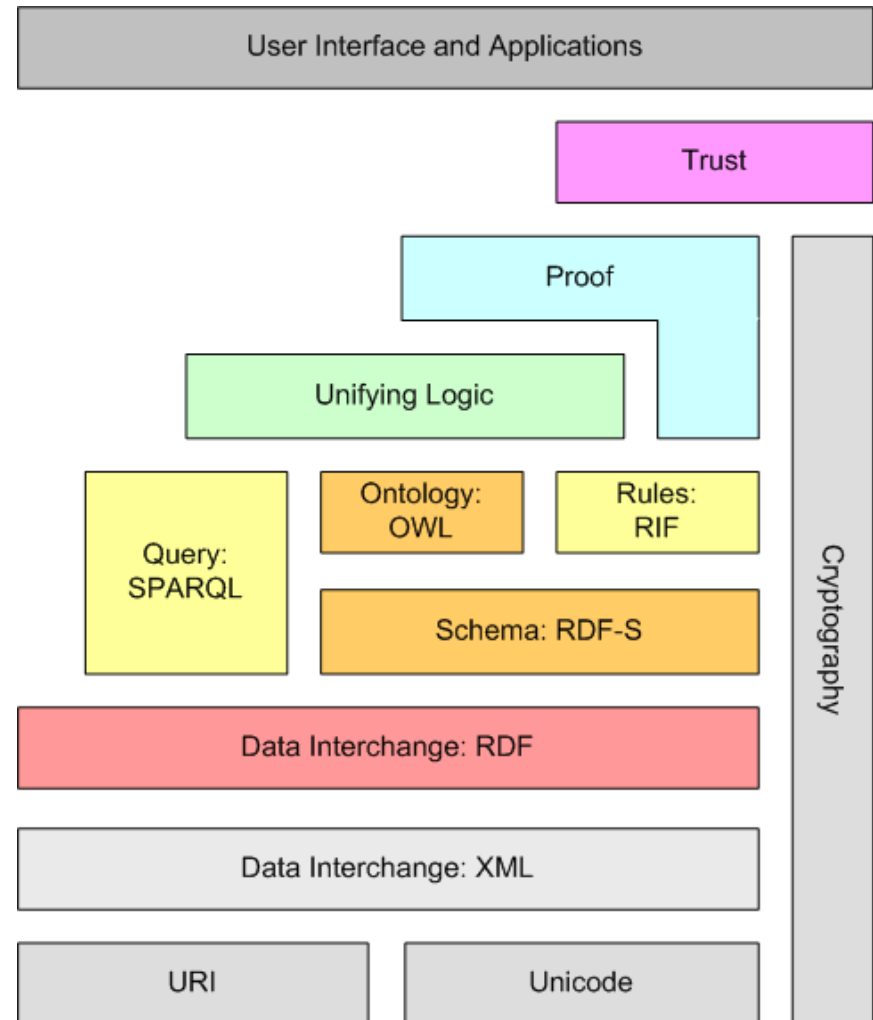
Semantic Web Technology Stack



here be dragons...

Knowledge Graph Technologies
(This lecture)

Technical
Foundations



Questions?

